

Software Testing Report

ENG1 Cohort 1 Team 3

Almira Demir

Mansi Deshkar

Laina Ha

James McDermott

Harry McLean

Vadim Mitko

In our team 3's version of UniSim, we utilized 3 different types of testing:

1. *Unit testing*
2. *Integration testing*
3. *Usability testing*

The project makes use of a headless test build that is built every time alongside the main build. It contains all the unit tests used by the main managerial classes in the game and integration tests for assets.

Unit and integration tests are automated, while functional testing has been performed by a few potential users who tested out our game during user evaluation in the last week of semester 1.

Unit testing is necessary to check whether each logically separate blocks of code are performing as expected and whether they can be used as building blocks for later code. If unit tests fail, the classes that are being tested can be reckoned to be unreliable. Unit testing is appropriate in UniSim as we have a few main Manager classes that interact with each other. To isolate errors that may appear in one of those classes, we can write individual unit tests for their methods. This will make it possible to isolate errors.

Integration testing is used by our team to check whether assets used in the game are being loaded as intended. The integration testing can save considerable time by first checking whether the assets are loaded correctly before any other logic related testing is checked. For example, if bakery assets are not being loaded in the intended manner, an integration test will point it out, saving time for us trying to figure out whether it could be faulty game logic that is causing this error or not.

At last, we have also used once usability testing to check whether the user interface and game logic made intuitive sense to a potential user. Testing has been performed in the last week of semester 1. During this session, we gave the user complete freedom to roam the game and try to figure out how to play it, simulating the way the user would interact with the game when they download and launch it for the first time by themselves. Usability testing is relevant for our project because it is able to point out any usability problems along with unmet user requirements. Usability testing was performed manually, as it is complicated and unreasonable to have it automated, as the

tests written for user interface will inevitably be opinionated and removed from actual user experience.

In our game, we have used unit testing to test whether managerial classes work as intended. We have written unit tests for the following classes:

- EventManager.java
- BuildingManager.java
- ScoreManager.java
- Timer.java

And we also wrote tests for assets in AssetTests.java

Event Manager

In tests for Event Manager, we have tested whether getRandomEvent() returns an object of type Event and that it is being correctly deleted from the events array.

Building Manager

In BuildingManagerTests.java, we have written tests to check whether placing down a building works as intended, meaning the building is added to buildings array, rendered on the map, and the building count is successfully updated. Moreover, we have also tested that helper getter methods work as intended. This means that after placing buildings, the map that maps building types to their counts is also being updated.

ScoreManager

When writing tests for ScoreManager, we made sure that the functionality of decrementing score with time is working correctly, alongside with the fact that the score will change after some combination of buildings is placed.

For example, testUpdateScoreWithBuildingCount() simulates how buildings are placed in the actual game and checks whether the calculated score matches the expected score, i.e. the bigger the difference in building type counts, the less is the score increase.

testDecrementScoreWithTime() simulates a timer going down, so in this test the score is expected to go down naturally over time.

Timer

In TimerTests, we have tests to check that whenever Timer ticks, the time is being decremented, and that Timer correctly updates whether it has finished ticking or not. Moreover, we also check whether getTimeToString() method correctly returns a formatted string that is ready to be used in our UI for the game

Assets

We also test for asset availability in AssetTests.java, which checks which building assets are present.

Latest Build test results

Package io.github.unisim.headless

all > io.github.unisim.headless

10
tests

0
failures

0
ignored

3.385s
duration

100%
successful

Classes

Class	Tests	Failures	Ignored	Duration	Success rate
AssetTests	1	0	0	2.774s	100%
BuildingManagerTests	3	0	0	0.412s	100%
EventManagerTests	1	0	0	0.012s	100%
ScoreManagerTests	2	0	0	0.147s	100%
TimerTests	3	0	0	0.040s	100%

Generated by [Gradle 8.10.1](#) at Dec 29, 2024, 7:48:58 AM

As can be seen from the above screenshot, the latest version of UniSim successfully passes all the tests. This means that:

1. Buildings are being placed correctly and related statistics about them are being updated correctly
2. Events happen in a correct manner and they don't appear twice during the gameplay
3. Score naturally decreases overtime and is being updated correctly when buildings are placed
4. Timer ticks correctly and is a reliable indicator of the state of the game

5. Assets for different buildings are present and being loaded correctly

Although our game passes all of the tests, we cannot claim that the automated testing that we have is complete, as some parts of the game still go untested automatically. For example that UI works as expected or that achievements and leaderboard work correctly. Present implementation of tests does not cover these relatively important areas of the game and the game developers in our team would benefit considerably if these tests would be automated.

However, to enable all such tests to be passed, classes used to implement leaderboard and achievements must be rewritten in a more complete and modular manner, like the “Subscriber-Publisher” pattern. This will allow for a better testing implementation, as unit testing aims to test the smallest parts of code.

If the classes are not modular enough, not all unit tests can be written, as non-modular classes also require methods and fields from other classes. Methods that lack modularity eliminate the purpose of unit testing all together. Moreover, the classes that are being tested above also would benefit from being more modular than they currently are. This blind spot of testing is definitely an area of improvement.

Also some classes like Timer are written in such a way that they are dependent on the game to function and are not standalone classes that can function in an isolated environment (like for unit testing). To be more clear, for example, the formatted string method currently only works in the way that works out the string that will be only displayed correctly if it is displayed in the UI of our game. If you decide to use it elsewhere, it would not function as intended.

Again, this problem stems from the low modularity of the implemented code and this is the issue that needs to be tackled first to improve the automated testing used in our game.